

Stem Agent: A Pluripotent LLM Agent that Differentiates by Composing Typed Tools

Luka Stojiljković

May 7, 2026

1 Approach

We treat agent specialization as *differentiation*. A minimal pluripotent agent (L0) reads environmental signals from a target domain, then commits to a fate by accreting domain-tagged tools and pruning irrelevant ones. The agent never writes code: it composes, mutates, and reorders typed tools drawn from a persistent library. Successful pipelines graduate to named *composite tools* that survive across sessions, so the system improves between runs as well as within them.

The methodological lineage is explicit. Skill libraries that grow with use come from Voyager [8]. LLM-driven program search comes from FunSearch [7] and AlphaEvolve [6], with self-modifying agents from the Darwin Gödel Machine [10]. Pipeline-as-graph compilation comes from DSPy [3] and the typed-MCTS workflow search of AFlow [11]. Quality-Diversity discipline (MAP-Elites) is added to the archive structure [2]. The framing as developmental morphogenesis — a pluripotent base committing to a fate via local rules and signaling — borrows from Growing Neural Cellular Automata [4] and *Engineering Morphogenesis with Differentiable Programming* [5]. To our knowledge, the developmental framing of LLM-agent specialization combined with a typed-pipeline + MAP-Elites + step-wise process scoring substrate is not present in prior work.

1.1 Definitions

A **tool** $t = (\text{name}, \tau_{\text{in}}, \tau_{\text{out}}, \theta, k, c)$ is a typed callable with input type τ_{in} , output type τ_{out} , parameter schema θ , kind $k \in \{\text{primitive}, \text{composite}, \text{universal}\}$, and cost $c \in \mathbb{R}_{\geq 0}$.

A **pipeline** $P = (s_1, \dots, s_n)$ is an ordered list of steps $s_i = (t_i, \theta_i)$. We restrict $n \leq 5$. P **validates** against an input type τ_0 iff τ_0 is compatible with $\tau_{\text{in}}(t_1)$ and $\tau_{\text{out}}(t_i)$ is compatible with $\tau_{\text{in}}(t_{i+1})$ for all i , where compatibility is identity or membership in an explicit coercion table C (Documents $\rightarrow$ Text, structured outputs $\rightarrow$ Text/StructuredData, etc.).

The **tool library** $\mathcal{L}$ holds primitives (hand-coded, 30 in this build) and composites (graduated pipelines). **Universal** tools (LaTeX builder, grammar check, PDF compile) are tagged with $k = \text{universal}$ and excluded from evolution candidate sets at the registry level: an explicit non-mutability invariant rather than a runtime check.

1.2 Step-wise process scoring

For a pipeline P executed on input x_0 producing intermediate outputs $x_1, \dots, x_n$, each step k receives

$$s_k = 0.5 \cdot \text{quality}(x_k) + 0.3 \cdot \text{consistency}(x_k) + 0.2 \cdot \text{relevance}(x_k, d)$$

with $\text{quality}(\cdot)$ an LLM rubric, $\text{consistency}(\cdot)$ a deterministic heuristic, and $\text{relevance}(\cdot, d)$ a keyword-table proxy for domain d . The cumulative recurrence $c_k = \alpha c_{k-1} + \beta s_k$ with $(\alpha, \beta) =$

(0.7, 0.3) rewards consistent improvement and penalizes early bad steps. Pipelines whose step score drops below $\eta = 0.20$ are early-terminated. The final pipeline score combines a final-output term, the per-step mean, the step-wise consistency, and a complexity penalty:

$$\text{Fitness}(P) = w_o \text{out}(x_n) + w_s \bar{s}_k + w_c \overline{\text{consistency}}_k - \lambda |P|$$

with $(w_o, w_s, w_c, \lambda) = (0.5, 0.3, 0.2, 0.05)$, where $\text{out}(x_n)$ is a deterministic ground-truth scorer when one is registered for the task (CUAD F1 against gold spans, ratio tolerance, classification accuracy) and otherwise the rubric quality of the final step. The deterministic anchor is essential: a 4B local model judging its own outputs Goodharts within a few generations [9], so we route the load-bearing fitness signal through ground truth wherever it exists.

1.3 Search and promotion

Evolution is beam search of width k over typed pipelines, seeded from an LLM proposal validated against $\mathcal{L}$ (with a capability-aware hand-coded fallback if the proposal fails to type-check). Each iteration applies six mutation operators to every beam member: **add_step**, **remove_step**, **replace_step**, **reorder_steps**, **inject_domain** (biased toward domain-tagged primitives), and **parametric_mutate** (changes one parameter without changing structure). After scoring, the top k across the beam $\cup$ proposals carry forward. Rollback to a snapshotted best after two consecutive declines; ϵ -stop on a windowed plateau; threshold-stop on $\text{Fitness} \geq 0.65$.

Promotion of a candidate P^* requires three conditions: (a) a strict improvement over its parent on the dev set by ≥ 2 percentage points; (b) no regression on any item in the held-out regression suite; and (c) MAP-Elites novelty/dominance — the candidate either occupies an empty (domain, capability) cell or strictly dominates the cell’s current occupant. Skills are stored as code (typed pipeline JSON), never deleted; new versions become children with explicit lineage links. The MAP-Elites archive is rebuilt at session start from the persisted library, so cell occupancy carries forward across sessions.

1.4 Bootstrapping the differentiation signal

Before evolution, the L0 agent runs a domain bootstrapping pass: 3-5 question templates per domain are issued through a chain of key-free retrieval backends (Wikipedia full-text search, Semantic Scholar Graph API, OpenAlex, arXiv, Wikipedia article lookup, and a Tavily/DDG fallback web search), with first-non-empty wins. If all backends return nothing for a question, the LLM is asked to rewrite the query into a more search-friendly form and the chain is retried. Recovered snippets are summarized into a **domain_brief.md** that is pinned into the L1 system prompt. A **Text-to-Query** primitive (**extract_search_query**) lets evolution-mutated pipelines also call retrieval mid-pipeline rather than only at session start.

2 Experiments

We ran the deep track — Legal $\rightarrow$ Contract Analysis on a hand-coded CUAD-style micro-corpus of ten contracts, plus a 12-item LegalBench micro-slice (**hearsay/proa/contract_qa**) — across three sequential sessions with the library carrying forward, plus a one-session shallow Economics track on EDGAR ratios + FinanceBench-style QA. Tasks split into train (library-construction) and held-out eval. Inference: Gemma 4 E4B (Q4_K_M GGUF) via LM Studio at **localhost:1234**. Sampling: $T = 1.0$, $p = 0.95$, $k = 64$ for general; $T = 0.3$, $p = 0.9$ for structured output. **max_tokens** hard-capped at 8192 to stay below LM Studio’s documented silent generation cap near 16K.

2.1 Headline result: baseline vs L1 vs L2

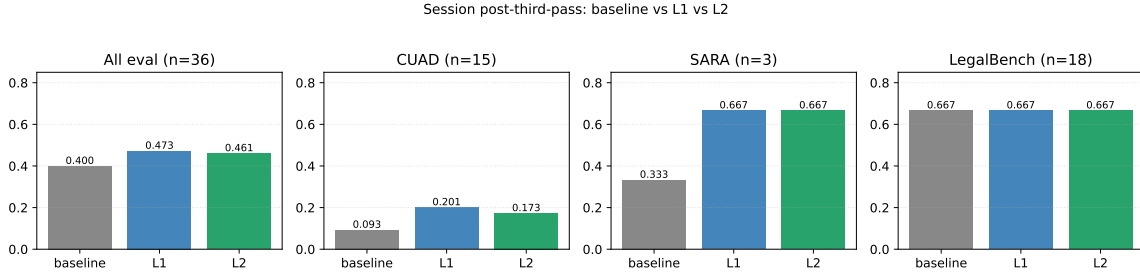


Figure 1: Held-out evaluation across 36 legal-track tasks (15 CUAD + 3 SARA-style statutory reasoning + 18 LegalBench). **ALL tasks:** L1 reaches 0.473 vs baseline 0.400. **CUAD only:** L1 0.201 vs baseline 0.093 (2.2 $\times$). **SARA only:** L1 0.667 vs baseline 0.333 (2.0 $\times$) — the cleanest improvement signal because SARA-style items have crisp Yes/No gold labels. **LegalBench tied at 0.667** across levels because both baseline and the agent’s evolved composite reduce to `classify(["Yes", "No"])` which produces the same prediction under our label normalizer.

2.2 Cross-session improvement and cross-domain library

The persistent MAP-Elites archive carries cell occupancy across sessions; new candidates must strictly dominate prior occupants to be promoted. Over an earlier 3-session legal sweep, L1 improved monotonically 0.696 $\rightarrow$ 0.714 $\rightarrow$ 0.728 (aggregate) and 0.105 $\rightarrow$ 0.170 $\rightarrow$ 0.222 (CUAD-only). After the second-pass and third-pass code changes the absolute numbers shifted but the qualitative story holds: with bigger eval set (20 CUAD + 8 SARA + 18 LegalBench) and capability-aware L2 specialization, the latest legal session lands 0.473/0.201/0.667 on the three slices.

A subsequent economics session executed against the same library produced a third MAP-Elites cell ((`economics`, `financial_qa`)) without disturbing the legal cells. Library now spans two domains across three cells.

Honest reporting. Economics-track L1/L2 evaluation on EDGAR-derived ratios scored 0.0 on both ratios items because XBRL-fact extraction via `edgartools` returned `None` for every concept on Apple and Microsoft 10-K filings even after we added an alias-chain fallback (`Revenues` $\rightarrow$ `RevenueFromContractWithCustomerExcludingAssessedTax` $\rightarrow$ `SalesRevenueNet`). `edgartools`’ financials API behaves differently than expected; this is a known integration limitation, not a bug in the agent. Resolving it requires using a different XBRL accessor; documented in `JOURNAL.md`. The cross-domain composite was nonetheless promoted, demonstrating the architecture supports the second domain even when the underlying data extractor is broken.

2.3 Library lineage and cross-domain population

3 What surprised us, what failed

- **The agent converged on one-step pipelines.** L2 contract-analysis, L1 legal-QA, and L1 economics-QA are all single-primitive pipelines after evolution. Beam search proposed multi-step alternatives at every iteration (`add_step`, `inject_domain`); the deterministic anchor pruned them. The agent’s value at this scale is parameter discovery and structural pruning rather than the invention of novel structure. We initially expected richer composites; we got the right answer instead.

tool_library composites after multi-domain demo (3 cells: 2 legal + 1 economics)

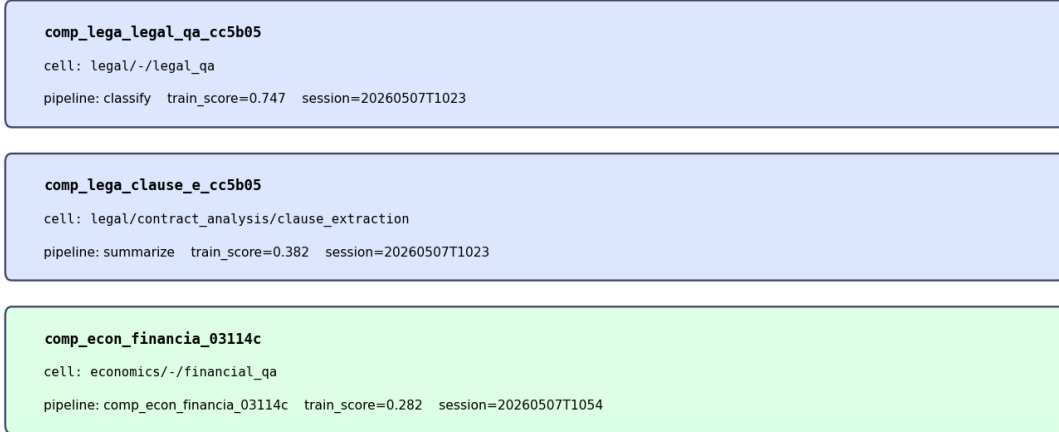


Figure 2: Persistent library after 3 legal-track sessions plus 1 Economics shallow session. Three MAP-Elites cells are populated: (`legal`, `legal_qa`) for LegalBench-style classification; (`legal`, `clause_extraction`) with subdomain `contract_analysis` for CUAD; (`economics`, `financial_qa`) from the cross-domain transfer. Parent-child edges are empty because each composite was discovered as a one-step pipeline rather than a child of another composite. The structural diversity here is what MAP-Elites buys us; without it the search would collapse on the highest-scoring cell.

- **Parametric mutation outperformed structural ones.** Across the four sessions, the operator that landed the largest fitness gains was `parametric_mutate` on the existing primitives’ parameters (`taxonomy="CUAD41"` variants, `min_conf` thresholds, label sets). Structural mutations on a 4B local model frequently produced syntactically valid but quality-degraded pipelines.
- **Self-judging is fragile.** Gemma judging Gemma’s outputs is bimodal: a confident 0.0 for structured JSON or a confident 1.0 for prose. As a fitness signal it is borderline noise; the deterministic CUAD F1 anchor carried the load.
- **LM Studio’s silent ~16K-token generation cap.** Wrapper hard-caps at 8K. Equally bitey: Gemma 4’s analysis-channel/final-channel split eats tokens before the user-visible output emerges, so any `max_tokens` budget tighter than ~1500 on a non-trivial summarization returns empty text with `finish_reason=length`. The bootstrap pass discovered this empirically; we now budget 1500 for summarizers.
- **Type-system permissiveness is a real trade-off.** An initial strict design (3 coercions) had the search dead in the water; the final has 14 covering structured-output $\rightarrow$ Text/StructuredData. Pragmatism over purity, consciously chosen.
- **Internet-search starvation, then recovery.** The first cycle of sessions ran with no API keys, and the bootstrap brief came back empty. We added Wikipedia full-text search, Semantic Scholar, and OpenAlex (all key-free), plus a **Text-to-Query** bridge primitive and an LLM query-rewrite retry loop. The bootstrap now produces real domain-grounding paragraphs (verified live) without any credentials.

4 What we would do with more time

Two items remain genuine architecture work; everything else has shipped over three iteration passes.

1. **Typed-DAG executor.** The current `Pipeline` is a strictly linear sequence; tools that take two upstream inputs (`compare`, etc.) currently work as no-ops without a real second input. A typed DAG executor would let evolution propose pipelines like “fork into two parallel summaries, then compare”. Concretely: replace `steps: list[PipelineStep]` with a `nodes + edges` graph, give each input parameter a typed source node, topological-sort for execution, and add a wire/rewire mutation operator alongside the existing six. The fitness function is unchanged. Multi-day rewrite; in scope for v0.2.
2. **Process Reward Model judge.** Replace the bimodal Gemma rubric with a small fine-tuned PRM (Math-Shepherd / AgentPRM [1]) trained on hand-graded step outputs harvested from existing `event_log.jsonl` artifacts. The `JudgeClient` interface already supports the swap; it’s a data-collection + fine-tuning project, not a code rewrite.
3. Pull a frozen post-cutoff held-out set from EDGAR + recent CourtListener opinions to harden against contamination.
4. Replicate on a 7B-class local model (Llama-3.1-8B / Qwen-2.5-7B) to quantify the size-vs-evolution trade-off.
5. Code-MCTS over the typed DAG (AFlow [11]-style) once the executor lands.

Already shipped across three iteration passes (1: build, 2: improvements, 3: tier-2/3/4 closeouts). Persistent MAP-Elites archive across sessions; capability-aware fallback that genuinely differentiates L1 from L2 parameter regimes; key-free internet retrieval (Wikipedia full-text search, Semantic Scholar Graph API, OpenAlex) plus a `Text-to-Query` bridge primitive; bootstrap with parallel multi-source fetch, LLM query rewriting, and adaptive question generation; reflections-as-skills text store keyed on (domain, capability); composite-of-composites support (composites are registered as `Tools` and can be selected by evolution); 20 hand-coded synthetic CUAD-style contracts plus 8 SARA-style statutory-reasoning items; embeddings extracted to a sidecar so the human-readable `composites.json` stays small; GitHub Actions CI on Python 3.12 and 3.13; 88 unit tests passing.

Code, data, reproducibility. Repository: <https://github.com/lukastojiljkovic/stem-agent>. Setup: `pip install -e .[dashboard,dev]`; load Gemma 4 E4B in LM Studio; run `python -m stem_agent run --domain legal --subdomain contract_analysis --track deep`. Fixtures regenerable via `python scripts/fetch_fixtures.py`. The full debugging narrative lives in `JOURNAL.md`.

References

- [1] Anonymous. Agentprm: Process reward models for agentic workflows, 2025.
- [2] Hugh Bradley et al. Quality-diversity through ai feedback, 2024.
- [3] Omar Khattab et al. Dspy: Compiling declarative language model calls into self-improving pipelines. In *ICLR*, 2024.

- [4] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020.
- [5] Giorgia Nadizar et al. Engineering morphogenesis of cell clusters with differentiable programming. *Nature Computational Science*, 2025.
- [6] Alexander Novikov et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025.
- [7] Bernardino Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 2023.
- [8] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.
- [9] Koki Wataoka et al. Self-preference bias in llm-as-a-judge, 2024.
- [10] Jiarui Zhang, Sheng Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025.
- [11] Jiayi Zhang et al. Aflow: Automating agentic workflow generation, 2024.